Compiling for Distributed Memory: A Historical Survey and Taxonomy from Global Arrays to Mesh-Sharded AI Systems

Sameeran Joshi

***Abstract*—Compilers that target machines with physically distributed memories must decide where data lives, where computation runs, and what communication connects the two. These decisions were posed as compiler problems in the late 1980s for hypercubes and multicomputers, standardised in High Performance Fortran in the 1990s, largely left to hand-written message passing in the 2000s, and have returned in the 2020s as sharding propagation for accelerator meshes and wafer-scale machines. This article surveys that history with a reproducible method: an operational definition of a distributed-memory compiler, an openly maintained and schema-validated catalog, and a taxonomy whose decision-procedure axis separates directives, closed-form analysis, analytic optimisation, and profiling or search. We coded the 54 catalog papers whose full text was read twice, once by the authors and once by an independent coder who had not seen the first coding. Agreement is low on most axes, so only the patterns that hold under both codings are reported as findings. Those patterns are that search and profiling enter the decomposition only from 2019, that message count led the objectives of the 1990s and is nearly absent later, that block-cyclic and mapping-array decompositions do not appear after 2010, and that partition propagation appears only in the 2020s beside owner-computes. Where ownership is set by an explicit map, the read systems that describe a run-time mechanism for it have a run-time stage, with two named exceptions. We list the earlier patterns that the second coding withdrew, and the open problems that remain, including the comparison of measured and modelled cost that no read paper makes.**

***Index Terms*—Distributed-memory compilers, data distribution, owner-computes, communication generation, High Performance Fortran, inspector–executor, polyhedral compilation, SPMD partitioning, tensor sharding, wafer-scale compilation.**

I. Introduction

A machine whose memory is distributed forces a compiler to answer three questions that a shared-memory compiler can defer to hardware: where each datum is stored, which place executes each operation, and what messages must flow so that operands are present when they are needed. Compilers that address these questions for hypercube and multicomputer targets appear in the 1988 literature [1]–[3]. Within five years the answers had been standardised in the directive language High Performance Fortran (HPF) [4], and the message-passing interface MPI had been standardised as the portable alternative in which the programmer answers all three questions by hand [5]. We observe that the same three questions are now being answered again, by SPMD partitioners such as GSPMD [6] and PartIR [7], by distributed tensor-algebra compilers such as DISTAL [8], and by compilers for wafer-scale and spatial accelerators [9], often with terminology that does not acknowledge the earlier work.

Manuscript draft generated from the open catalog at https://github.com/Sameeranjoshi/dist_compilers. This is a working draft; see the repository ROADMAP for status.

A. Why the problem keeps reappearing

Every time a new class of machine with distinct memories becomes important—hypercubes, clusters, multi-GPU nodes, accelerator pods, wafers—the trade-off between programmer control and compiler automation is re-opened, and the compiler community rebuilds a stack of ownership representation, computation mapping, and communication synthesis for that class. The repository that accompanies this survey exists to make that rebuilding less amnesiac: it records, with evidence, which mechanism solved which problem, under what assumptions, and with what limitation.

B. Scope

We define a *distributed-memory compiler* as a compiler, compiler framework, programming system, or compiler-integrated runtime that transforms a program for execution across multiple computational places whose memories are physically or logically distinct at the relevant compilation level (Section II gives the inclusion criteria). Multiple GPUs are a distributed-memory target when the compiler must partition data or computation and generate communication between device memories; wafer-scale and spatial architectures count when individual processing elements have distinct local memories and data movement must be compiled explicitly. Distributed databases, distributed operating systems, cloud schedulers, hardware-only papers, and shared-memory-only compilers are out of scope.

C. Central question and research questions

The central question is: *how have compilers for machines with physically distributed memories represented, decomposed, mapped, communicated, optimized, and executed data and computation?* We refine it into ten research questions (RQ1–RQ10) on programming abstraction, data decomposition and ownership, computation decomposition, communication generation, the compiler/runtime boundary, compiler representation, optimization objectives, evaluation, historical continuity, and open problems. Section V maps each to an axis of the taxonomy.

D. How to read this survey

A reader new to the field should read Sections IV and VI in order; the era narratives are self-contained and each ends with what it left to the next. A reader who knows one era and wants the connections should go to Table I in Section XII, which pairs each mechanism with its earlier or later form and cites the cards on both sides. A reader who wants to challenge a claim should use the repository: every count is regenerated from the catalog, and every card names the sections it rests on.

E. Contributions

1) An operational definition and inclusion criteria for distributed-memory compilers that span 1950–2026 while keeping pre-1980 work explicitly labelled as *foundations* rather than as distributed-memory compilation (Section III).
2) A reproducible survey methodology with an open, schema-validated catalog in which every entry carries evidence pointers and a screening status (Section II).
3) A common architecture for distributed-memory compilers (Section IV) and a multi-axis taxonomy (Section V), both derived from the catalog.
4) A history in six eras, each told in the same five parts: the machines and the programmer's problem, the central idea, the design space in which the era's systems differed, what the era learned from its own measurements, and what it left unsolved (Section VI); and mechanism sections that place each system on the taxonomy's axes (Sections VII–XI).
5) A then-and-now analysis that pairs eight mechanisms of the 1990s with their 2020s forms at the level of mechanism, with evidence from both sides; six patterns counted across the read papers; the four causes of HPF's decline applied to today's sharding compilers; and eight generalizations each stated with its evidence strength and the observation that would refute it (Section XII), with the open problems they leave (Section XIII).

[TODO: We do not claim that this is the first comprehensive survey of the area; an explicit search for prior surveys is logged in the repository and the claim will be revisited once it is complete. Known prior overviews include [10]–[12].]

II. Survey Methodology

A. Scope and inclusion criteria

A system is included when it performs or represents at least one of: data distribution or ownership; computation partitioning or placement; communication generation; collective insertion; halo or ghost-region construction; global-to-local address conversion; import/export schedule generation; redistribution or resharding; hierarchical parallelism across nodes or devices; distributed execution-plan generation; or distributed-memory performance modelling used by a compiler. We classify included work as *central* (automatic distributed-memory parallelization; global-array and data-parallel compilers; Fortran D, HPF and relatives; automatic message-passing generation; distributed dense and sparse tensor compilers; SPMD partitioners; distributed GPU/accelerator compilers; spatial and wafer-scale compilers) or *bridge* (languages whose compilers implement distributed memory; compiler-integrated runtimes such as inspector–executor systems; PGAS compilers; task/region systems where placement is central; distributed-training systems with substantial plan generation; performance models that drive compiler decisions). Excluded are distributed databases and operating systems, generic job scheduling, routing protocols, hardware-only papers, application scaling studies, shared-memory-only compilers, single-GPU kernel compilers, and pure runtimes without a compiler component. A small number of architecture papers are retained in the catalog solely to date target classes; they are not classified on the compiler axes.

B. Sources and queries

Bibliographic metadata were taken, in order of preference, from the DOI record (via Crossref), arXiv, official author or institution copies, official project documentation, and trusted bibliographic indexes. No paywall was bypassed; where full text was not available the catalog marks the entry `paywalled` or `abstract-only` and no mechanism is inferred from the title. The initial query set (25 phrases such as "data distribution compiler", "communication set generation", "inspector executor distributed memory", "SPMD partitioner compiler", "wafer scale compiler") is listed in the repository together with a discovery log recording each query, date, hits, and screening decision. [TODO: Report per-database hit counts once the full query sweep over ACM DL and IEEE Xplore is logged.]

C. Snowballing

From every anchor paper we snowball backward (references), forward (citing works, prioritising those that extend, critique, evaluate, or revive the anchor), laterally (same system, project, or authors), across surveys (works cited by several independent surveys are prioritised), and by *gap*: for every empty cell of the taxonomy coverage matrix we search for that combination specifically.

D. Screening and evidence levels

Each catalog entry carries one of five statuses: *discovered* (metadata only), *screened* (abstract and introduction checked), *core* (clearly relevant), *read* (enough read to summarise accurately), and *verified* (metadata, categorisation, and summary reviewed against the primary source). Only *read* and *verified* entries support mechanism-level claims in this article. A paper is called a survey only if it is a survey or an invited overview; for example, Lee's "Techniques for compiling programs on distributed memory multicomputers" [13] is catalogued as a technique paper.

E. Reading strategy and paper cards

For most papers we read the abstract, introduction, system overview, related work, and conclusion; we read the full paper when it introduced a major system, defines a taxonomy category, supports a key historical claim, has experiments that we compare, or supplies a mechanism central to the synthesis.

Every core paper receives a paper card in the repository listing problem, input abstraction, target, decomposition, mapping, communication, compiler/runtime split, evaluation, limitation, lineage, and the exact sections used.

F. Evidence labelling

Statements in this article are of three kinds: source-supported facts, which carry a citation; survey taxonomy introduced by us, flagged by "we categorize" or "we group"; and inferences, flagged by "appears", "we observe", or "we hypothesise" and listed with their current evidence in the repository's historical-claims file. Conflicts between sources are recorded rather than resolved silently.

G. Limitations

Full text of many 1988–2000 papers is paywalled; their summaries rely on open author copies where available. Citation counts are not used as a relevance criterion. Dates of era boundaries are provisional and are justified by the dated events listed in the catalog.

III. Foundations (c. 1950–1979)

This section separates what the distributed-memory compilers of the late 1980s inherited from what they invented. Nothing here is a distributed-memory compiler. The machines of the period had a single address space or lock-step SIMD control, and placing data in memories was not a compiler concern. Where a foundations paper has no card in the repository yet, we state only its bibliographic record and mark the content claim as pending.

A. Automatic programming

The FORTRAN automatic coding system established that a translator could generate object code for numerical loops from a higher-level notation [14]. We take this as the first instance of the bargain that every later system renews: the programmer writes a higher-level program, and the compiler makes the low-level decisions. [TODO: Add a card for [14] and quote its code-quality claims only after reading.]

B. Dependence analysis and loop parallelism

Lamport's paper on the parallel execution of DO loops, published in 1974, is the standard reference for reordering nested loop iterations so that independent iterations run together [15]. Kuck and colleagues presented dependence graphs as the representation on which compiler optimisation and parallelisation are built [16], and Allen and Kennedy translated FORTRAN loops into vector form using the same analysis [17]. Distributed-memory compilers reused this dependence representation without change for the iteration and access analysis; what they added was an ownership function. Section VII shows this for the Fortran D and PARADIGM lineage. [TODO: Read [16] and [17] and replace this paragraph with card-backed statements.]

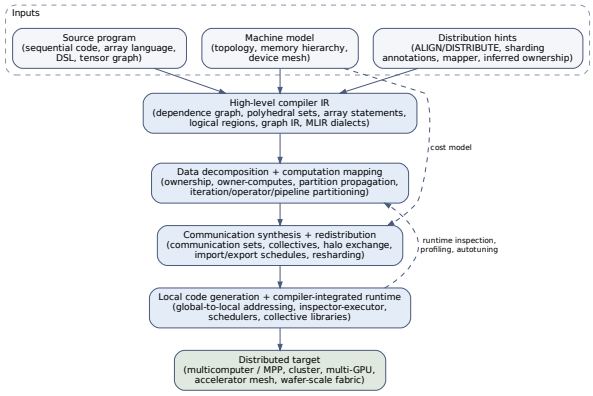


Fig. 1. The stages that recur across the catalog's distributed-memory compilers. Most 1990s systems are linear through the communication stage. The dashed edges are where systems differ on the compiler/runtime boundary (axis F): runtime inspection, profiling and cost models feed back into decomposition. The figure is original to this survey and is abstracted from the cited systems.

C. Machine abstractions

The ILLIAC IV array processor [18] and the CRAY-1 vector computer [19] are the two regular-parallel machine classes that the early loop compilers targeted. Dennis and Misunas proposed a dataflow processor architecture in 1975 [20], and a 1977 MIT workshop report on dataflow computers and program organisation is in the catalog from a supplied copy [21]. Kung's article on systolic architectures [22] and the Cosmic Cube multicomputer [23] establish the two architectural ancestors of the spatial and distributed targets discussed in Sections XI and VI. [TODO: Read [22]: the supplied copy has no text layer and must be read visually before any content claim is made.]

D. Data-parallel programming

Hillis and Steele described data-parallel algorithms on the Connection Machine, in which one operation applies to all elements of a data set at once [24]. This is the programming model that Fortran 90 array statements and, through them, HPF inherited. [TODO: Verify the lineage claim against the HPF specification [25] once that record is read.]

E. Synthesis

The foundations supplied three things to the next era: a trusted pipeline for transforming loops, a legality test for reordering them, and machine models with regular parallel structure. What they did not supply was any representation of where data resides. That omission became the central compiler problem of the late 1980s, once machines with private memories appeared [23].

IV. A Common Compiler Architecture

We abstract the systems in the catalog into the pipeline of Fig. 1. The inputs are a program (sequential code, an

array language, a domain-specific language, or an operator graph), a machine model (processor arrangement, device mesh, memory hierarchy), and distribution information. That information is either supplied by the programmer, as ALIGN and DISTRIBUTE directives [4], sharding annotations [6], or a task mapper [26], or it is inferred.

a) Intermediate representation: The representation determines which decisions can be made statically. Vienna Fortran's distributions are declarations attached to arrays [27], and Kali's distributions are declared with processor arrays and on-clauses [28]. Koelbel's closed forms are derived from the subscripts of forall loops [29], and Chatterjee et al. give a finite-state representation of local access sequences for cyclic(k) distributions [30]. The polyhedral framework represents iteration and access spaces as integer sets and was used for MPI generation by Bondhugula [31]. Mesh-TensorFlow names tensor dimensions and maps them onto a processor mesh [32], and MLIR's shard dialect represents grids and per-axis shardings as IR attributes [33].

b) Data decomposition and computation mapping: Kali's doall loops execute an iteration on the processor named by an on-clause, and the report describes owner-based placement [28]. Chen et al. compile Crystal programs, whose data fields carry communication metrics, by reshaping them with morphisms [3]. In GSPMD, sharding annotations are completed by propagation through the graph with priorities, and each operator is then partitioned [6]. Each approach derives a computation mapping from a data mapping, though the direction of derivation and the place where the decision is made differ.

c) Communication synthesis and redistribution: Communication can be derived in closed form from the distribution functions [29], [30], recognised as collectives, or determined at run time from index arrays [34]. Redistribution between distributions is handled by pairwise communication plans in Ramaswamy et al. [35], and by phase-level planning in Palermo et al. [36]. In MLIR's shard dialect, collectives such as all-gather, all-reduce and all-to-all are operations with the participants inferred from the sharding [33].

d) Local code generation and runtime: The PARTI/CHAOS line supplied schedules for off-processor accesses [37], [38]. Legion executes a task graph over logical regions and maps them to memories through a mapper [26]. Rogers and Pingali emitted per-process C code with run-time resolution as a fallback [39]. The amount of work delegated to the runtime is axis F of the taxonomy.

e) Synthesis: The pipeline has been stable since the early 1990s. What has changed is the input abstraction (axis B), the representation on which decomposition is decided (axis K), and which stages are static (axis F). These three differences are the axes along which the eras of Section VI separate.

V. Taxonomy

A chronological list cannot separate the systems this survey is about, because the same mechanism returns under different names in different eras. We therefore classify each paper on independent axes. Fig. 2 shows the axes, the catalog carries eleven, and each entry takes one or more values on each. The value lists are maintained in the repository and enforced by a validator.

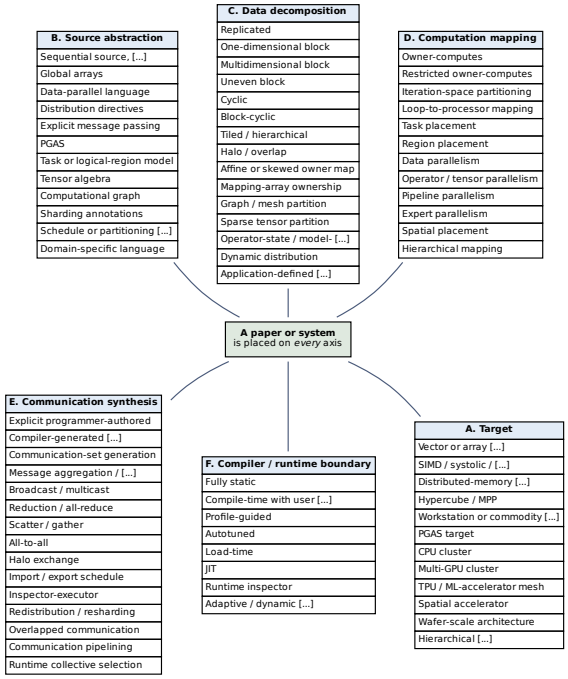


Fig. 2. The taxonomy as orthogonal axes rather than a tree. A paper or system takes one or more values on every axis. Only six axes are drawn here; the catalog carries eleven. The figure is generated from the machine-readable taxonomy so that it cannot drift from the catalog.

A. The axes

The axes answer different questions. Target (axis A) asks what machine the code runs on. Source abstraction (B) asks what the programmer writes. Data decomposition (C) asks how ownership is represented, and computation mapping (D) asks how work follows ownership. Communication synthesis (E) asks what messages or collectives the compiler produces. The compiler/runtime boundary (F) asks which decisions are taken when, and compiler architecture (G) asks how the compiler is built. Optimisation objective (H), maturity (I), paper type (J) and compiler representation (K) complete the set.

B. Refining the compiler/runtime axis

Our first coding of axis F was too coarse. It had one value, "fully static", for every system that decides without running the program. That value merged two different procedures. Kali's closed-form communication sets are obtained by analysis of subscripts. Gupta and Banerjee's distribution is chosen by minimising a weighted cost over constraints [40], and Lee's and Palermo et al.'s layouts come from dynamic programs over a cost model [13], [36]. Kong et al.'s mapping comes from a solver over a volume cost [41]. These are different decision procedures, and they behave differently when the

program changes. We therefore split the value into an analytic-optimisation value, and we record directives, profiling and search separately (Section XII gives the counts under both codings). The refined axis is the basis for the patterns in Section XII.

C. A value the axes did not have

Owner-computes assigns each statement to the owner of its left-hand side. Sharding systems do not assign statements at all: the sharding of a value is propagated through the graph, and each operator is partitioned according to the sharding of its operands [6], [7]. We added a partition-propagation value to axis D so that the two mechanisms can be compared rather than conflated.

D. Coding and its limits

The first coding was made from the cards: each entry's tags name the section that supports them. It was made by the authors and by agents drafting cards, and the authors checked the tags. A second coding was then made by an independent coder who had not seen the first. The coder received only the card text and the definitions of the five axes that the counts use: data decomposition, computation mapping, communication, the compiler/runtime boundary and objectives. Exact-set agreement between the two codings is 26% for data decomposition, 20% for computation mapping, 20% for communication, 52% for the compiler/runtime boundary and 6% for objectives. We therefore treat the objectives axis as unreliable, and we report a pattern only when its direction holds under both codings and under their intersection (Section XII). The agreement tables and the sensitivity analysis are generated from the repository (`docs/coding`).

VI. Historical Evolution

Each era below is told in the same five parts: the machines and the programmer's problem, the central idea the era bet on, the design space in which its systems differed, what the era learned from its own measurements, and what it left unsolved. The facts come from the paper cards; where the cards give a number we quote it, because the numbers explain the bets. Eras 0 and 1 have no full-text reads and are treated briefly. Section XII tests the patterns that emerge here against the whole read set.

A. Era 1 (c. 1980–1987): private memories become the programmer's problem

Message-passing multicomputers such as the Cosmic Cube gave each node a private memory and a message network [23], and the Connection Machine offered a data-parallel model in which one operation applies to every element at once [24]. The records of this era in the catalog are hardware and programming-model papers, not compilers. What the next era's authors say about this period is consistent: every one of the 1988–1992 systems opens by describing hand-written message passing as the problem to remove. [TODO: Add the early source-to-source placement systems of this period after backward snowballing from the 1988 papers.]

B. Era 2 (c. 1988–1992): global arrays and the owner-computes bargain

a) Machines and the problem: The targets were hypercubes and meshes: the iPSC/2 and iPSC/860, the NCUBE, SUPRENUM [29], [42]. The Kali report states that the environments of the time made the user encode every detail of data and code placement [28]; the Fortran D authors report that message-passing versions of a program grew three to ten times in size [43]; and the Vienna Fortran tutorial shows, with a Jacobi relaxation, that the hand-written version hard-wires one blocking into local arrays, overlap areas, guarded sends and receives, so that trying another blocking means rewriting all of it [42].

b) The central idea: Keep the global index space and sequential semantics, add a declaration of how arrays are distributed, and let the compiler derive the node program. Zima and Chapman describe the derivation in four phases: a front end, a split into host and node programs, an initial parallelisation in which every assignment is masked by an ownership test and every nonlocal reference satisfied by a per-element exchange, and an optimisation phase that replaces the per-element exchanges with overlap-area exchanges, hoists and combines messages, and removes the masks by shrinking loop bounds to each processor's own index set [10]. The rule that makes the derivation possible is owner-computes: the processor that owns the left-hand side executes the assignment, so the work set of a statement follows from the distribution. Kali expresses it directly, by running each doall iteration on the processor named in an on-clause, typically the owner of a named element [28], and Fortran D derives the local iteration set of each statement from its left-hand side [43]. The bargain is the FORTRAN bargain again: the programmer gives up control of placement in exchange for a program that is portable across blockings.

c) The design space: The systems differed on three questions. First, what the programmer declares. Kali has processor arrays, block, cyclic and block-cyclic distributions, user-defined map functions and a redistribution statement, but no alignment construct [28]. Vienna Fortran adds alignment with linear expressions, dynamic distributions with connect sets, INDIRECT mapping arrays and user-defined distribution functions, and lets a FORALL run on a named processor, on the owner of a datum, or where the compiler decides [27], [42]. Fortran D aligns arrays to a decomposition and then distributes the decomposition [43]. Crystal takes a different road: a functional language whose data fields carry communication metrics, reshaped by morphisms [3], and Rogers and Pingali start from an Id-like language with mapping functions [39]. Second, how much is decided at compile time. Kali computes its in and out communication sets symbolically when the distributions and subscripts admit closed forms, and otherwise runs an inspector, a modified copy of the loop that records nonlocal references, whose results are cached across repeated executions [28]. Koelbel proves closed-form send, receive and iteration sets for forall loops over block and cyclic arrays with linear subscripts [29]. The PARTI line takes the opposite end: a preprocessor builds paired send and receive lists at run

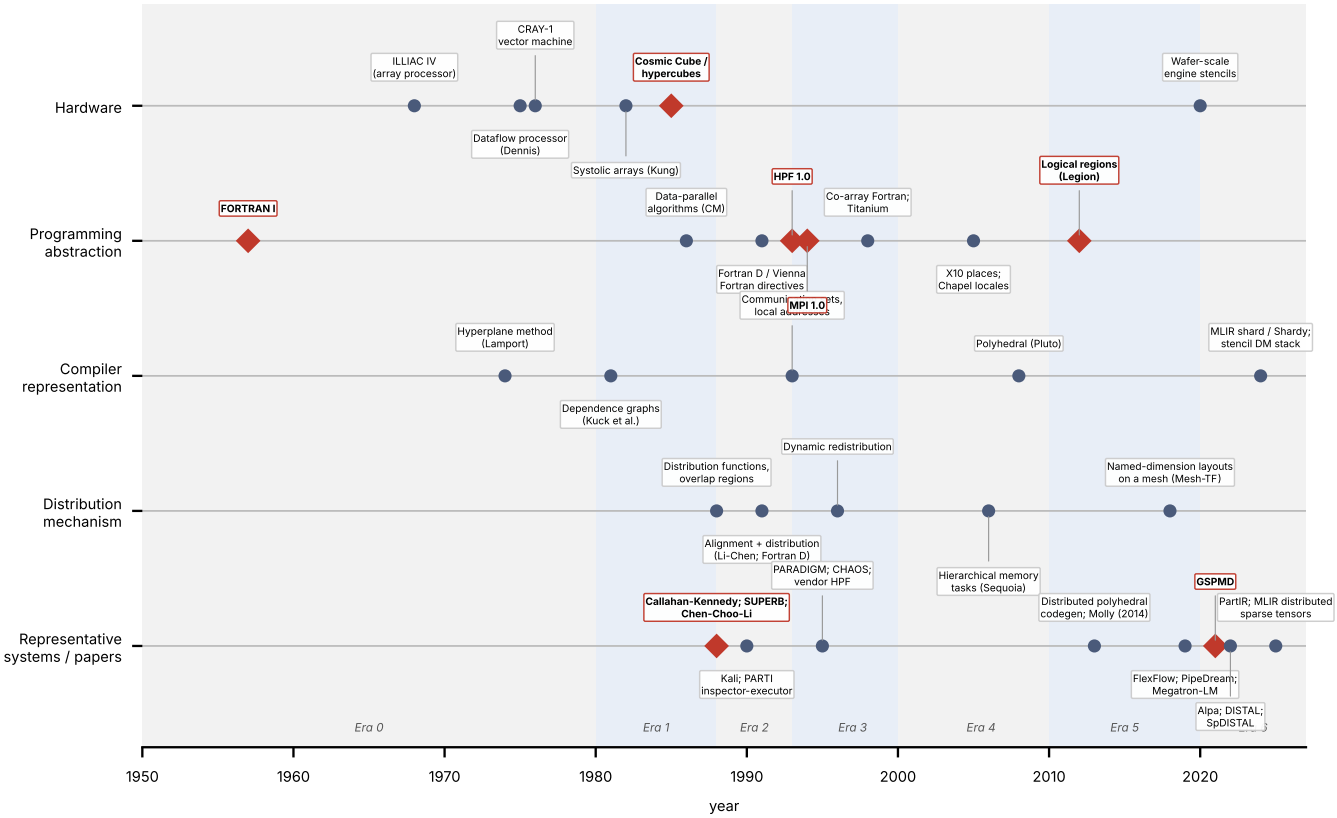


Fig. 3. Timeline 1950–2026 with lanes for hardware, programming abstraction, compiler representation, distribution mechanism, and representative systems. Diamonds mark transitions that open a new era. Each event is tied to a catalog entry; era boundaries are provisional.

time and a table-driven executor runs them [34], [44]. Third, whether the distribution itself can be chosen automatically. Gupta and Banerjee record, for each parallel loop, constraints on the distributions weighted by estimated penalties, and combine them into one distribution per array; they note that the problem is NP-hard and that their estimates assume known loop bounds and branch probabilities [40].

d) What the era learned: The one systematic evaluation in the read set explains the era's objective. On the iPSC/860, a message cost about 95 microseconds for its first byte and 0.4 microseconds for each further byte, and the Fortran D evaluation groups every optimisation by which term it reduces: startup, copy and transit, or lack of parallelism. Message vectorisation, which hoists element messages out of loops, gave two to nine times over element-wise messages on stencils [45]. A startup cost more than two hundred times the per-byte cost makes the number of messages, not their volume, the quantity to minimise. The era's evaluation attacks startup cost first, by its own classification [45]. The objective counts for this era differ between the two codings and are not used to say more. The closed-form path was shown to be cheap where it applies: Koelbel's generated cyclic-reduction code averaged 2.2% overhead against hand code on an iPSC/860, with the author's hand code as the baseline [29], while run-time resolution was expensive: Rogers and Pingali's compile-time resolution reached about three quarters of hand-written speed on Gauss-Seidel, and their run-time resolution was an order of magnitude slower still [39]. The Kali report itself says it is not clear a priori whether the run-time overhead of inspection is justified [28], and the PARTI measurements put the inspector at three to five iterations' worth of compute on an iPSC/860, dominated by translation tables [44].

e) What remained: Vienna Fortran's tutorial lists the competing proposals of 1992: the Yale extensions, Digital's HPF proposal, Cray MPP Fortran, Fortran D and Steele's amendments [42], and Zima and Chapman name the High Performance Fortran Forum as the body seeking a common basis [10]. Three things were unsolved: there was no common language; automatic distribution was NP-hard and untested beyond single kernels; and irregular codes needed a run-time stage whose cost was not yet known to pay. The first was addressed by standardisation, the other two were carried into the next era.

C. Era 3 (c. 1993–2000): standardisation and the limits of static compilation

a) Machines and the problem: Massively parallel processors (Paragon, CM-5, SP-1 and SP-2) and the first workstation farms [36], [46], [47]. Digital's compiler team describe the state of practice as hand-written SEND and RECEIVE code that they liken to assembly programming, and set the compiler the task of deriving SPMD code, local addressing and all communication from the declared layout, under the constraint

that a network access costs orders of magnitude more than a local one [47].

b) The central idea: A standard. HPF 1.0 added directives to Fortran 90: PROCESSORS, TEMPLATE, ALIGN and DISTRIBUTE with BLOCK, CYCLIC and CYCLIC(k), plus FORALL and INDEPENDENT; the directives are comments, so a sequential compiler can ignore them [4], [25], [48]. The retrospective by Kennedy, Koelbel and Zima records that Fortran D's two-level template-and-distribution scheme was adopted into HPF, and that block-cyclic was kept because it balances load on triangular computations such as LU decomposition [48]. Loveman, who edited the specification, states that the Forum simplified the design to reach agreement within a year, and lists irregular and user-defined distributions, non-rectangular processor arrangements and explicit message passing among the capabilities left out [4].

c) The design space: With the language fixed, the systems differed in compiler technology, and the cards show four lines of work. The first is exact address and communication-set generation for the whole distribution family. Chatterjee et al. showed that a processor's local access sequence for a regular section under cyclic(k) with affine alignment is produced by a finite-state machine with at most k states, and gave table-construction algorithms [30]. PARADIGM extended its symbolic framework from block to cyclic and block-cyclic distributions by delegating the Fourier-Motzkin elimination to Mathematica, so that loop bounds and communication sets are derived with the processor count left symbolic [49]. IBM's pHPF likewise kept the processor count as a symbol, so that one object code runs on any number of processors [50]. The second is the production compiler's pipeline. Digital's compiler builds a data space per symbol and an iteration space per expression, splits expressions into communication-free regions, materialises each inter-region transfer as an assignment to a temporary, and emits inlined addressing with explicit sends and receives and no barriers; it adds shadow edges for nearest-neighbour stencils and inlines reductions [47]. Benkner and Zima's overview describes the execution scheme that overlaps communication with computation: send first, compute the local part, receive, then compute the nonlocal part [11]. The third is the choice of distribution over time. Palermo et al. split the program into phases by repeated maximal cuts of a communication graph and choose the sequence of distributions by a shortest-path search that charges for redistribution [36], and Ramaswamy et al. represent each regular distribution as a family of line segments so that the data to move between two processor sets is an intersection of families [35]. The fourth is the irregular path. CHAOS lets the user name a partitioner that works from geometry, connectivity or load, builds the irregular distribution at run time, and reuses inspector results when a global modification counter and per-array stamps show that the indirection arrays are unchanged [51]; for adaptive programs it adds light-weight schedules that omit placement order [38].

d) What the era learned: The compilers were good on regular codes and the measurements say where they stopped. Digital reports speedups of 8.6 to 17 on eight processors for shallow-water, conjugate-gradient and red-black kernels, with the HPF red-black code faster than a hand-written PVM version [47]. pHPF matched hand-written MPL on one benchmark (30.5 against 30.4 on 32 processors) but reached 19.3 against 31.1 on another, and its authors state that compiled HPF carries inherent overheads that leave it slower than tuned hand code [50]. The CHAOS measurements show what schedule reuse is worth: an unstructured Euler solver on 32 processors took 18.5 seconds hand-coded, 19.8 compiled with reuse, and 301 compiled without it [51]. Benkner and Zima conclude that non-unit-stride alignment onto a block-cyclic target yields irregular local sections that need table-driven translation, and that removing such mappings from the language would allow a more efficient implementation without loss of needed function [11]. DEC's compiler did not adopt strict owner-computes: it chooses the iteration space of each expression to minimise communication, and gives an example where computing where the operands are aligned and moving one result is cheaper than moving two operands to the owner [47]. Two lessons follow. The closed-form machinery was complete enough to be shipped, but the richest part of the distribution language cost more than it returned; and the owner rule was already being relaxed by the compilers that took performance seriously.

e) What remained: Kennedy, Koelbel and Zima attribute HPF's loss of users to four causes: immature compiler technology combined with impatience in the community, insufficient support for important features, inconsistent implementations, and the complex link between a program and its performance [48]. Benkner and Zima add that irregular codes needed the HPF_LOCAL escape and that the data-parallel model was too limited for task parallelism [11]. The catalog's evidence supports the second and fourth causes directly: the features left out in 1993 are the ones the irregular applications needed, and the performance of compiled code depended on analyses the programmer could not see. Section XII returns to all four causes, because the same questions can be asked of today's sharding compilers.

D. Era 4 (c. 2000–2010): locality in the language, mapping outside the compiler

a) Machines and the problem: Clusters of commodity nodes, cache-coherent NUMA machines, and the first explicitly managed memory hierarchies. Chandra et al. report that on a cache-coherent NUMA machine a remote miss costs two to three times a local one and that the operating system places data in 16 KB pages, so data meant for different processors can share a page [52]. Sequoia's authors state that distribution languages address horizontal communication across a cluster but not movement through a memory hierarchy [53].

b) The central idea: Put locality in the language's types, and take the mapping decision out of the compiler. Co-Array Fortran asks for the smallest change that makes Fortran 95 a parallel language, and its authors say it separates work distribution from data distribution, which they state HPF entangles: a reference without brackets is local, a reference with a co-dimension in brackets is remote, and synchronisation is the programmer's responsibility [54]. UPC declares a block size on each shared array and tests affinity per iteration in

upc_forall [55]. Titanium builds distributed structures locally and exchanges pointers to form a replicated directory, so affinity is chosen by allocation [56]. Chapel separates the domain (the index set) from the array, lets a domain be distributed over locales with user-written distribution classes, and runs a forall iteration on the locale that owns its index unless an on-clause says otherwise [57]. Sequoia goes furthest: programs are trees of side-effect-free tasks, communication happens only through task arguments, and a per-machine mapping specification, separate from the source, binds tasks to memory levels [53].

c) The design space: The systems differ in how much the compiler still does. At one end, the PGAS compilers translate to C and leave placement to the program: the Berkeley UPC translator did no UPC-specific communication optimisation at the time of its evaluation [55]. In the middle, the Titanium compiler converts blocking copies to non-blocking ones, applies an inspector-executor to indirect patterns, and infers which pointers are local [56]. At the other end, the polyhedral and tiling work of the period keeps full automation but narrows the input: Tang and Xue assign tiles to processors first and let data follow by what they call the computer-owns rule, the reverse of the HPF order [58]; Baskaran et al. allocate scratchpad buffers and choose tile sizes from a cost model for explicitly managed memories [59]; and Shen et al. remove iteration variables that do not affect which element or processor is involved from the communication sets of Amarasinghe and Lam [60].

d) What the era learned: A remote access through a UPC shared pointer cost about two orders of magnitude more than a local one on the measured cluster, and the hand-applied transformations the authors used as upper bounds were message pipelining, software pipelining and combining reads [55]. Titanium's local-qualification inference alone gave speedups of 239% on CG and 443% on MG [56]. Sequoia ran one program on a Cell blade and an InfiniBand cluster with changes only to leaf kernels and the mapping file [53]. The optimisations the PGAS compilers needed are the Fortran D optimisations under new names: bulk transfer is message vectorisation, and non-blocking copies are overlap. What the language changed was who decides placement; what the compiler still had to do was the same.

e) What remained: Co-Array Fortran's paper reports no measurements and places ordering on the programmer [54]; Chapel's authors call execution-model transparency their weakest area and had no implementation to measure [57]; Sequoia's authors doubt that its primitives express dynamic applications whose accesses cannot be listed as task arguments [53]. The era kept locality visible and gave up automatic placement. Automatic placement returned in the next era by two different routes.

E. Era 5 (c. 2010–2019): exact dependences, runtime tasks, and the mesh

a) Machines and the problem: Clusters of multicore nodes with InfiniBand, GPU nodes, and the first accelerator pods. Bondhugula states that no practical end-to-end solution existed for compiling regular sequential programs to clusters, and names the obstacle: the process count is a run-time parameter, so the identity of communication partners is not an affine function [61]. Legion's authors state that misplaced data or computation is costly on heterogeneous processors with deep, distributed memory hierarchies [26]. PipeDream's authors measure communication stalls of up to 90% of training time at 32 GPUs for data-parallel training [62].

b) The central idea: Three ideas, from three communities, for the same decision. The polyhedral line removes ownership altogether. Bondhugula's generator is, in its authors' words, data-distribution free and computation driven: there is no owning processor, every process holds the full data space, the computation partition comes from Pluto's tiling, and the communication is computed from flow dependences as per-tile flow-out and write-out sets, with run-time helper functions that enumerate the receiving ranks once the process count is known [31], [61]. Molly puts the same exact dependences inside LLVM and Polly, with a home location for each array element and a where-mapping for each statement, and groups transfers into chunks [63]. Dathathri's partitioning of the flow-out set by receiver cut volume by 1.4 to 63.5 times over the unpartitioned scheme [64]. The task line moves the decision to a runtime. Legion's tasks declare privileges and coherence on logical regions, a software out-of-order processor finds the dependences among sibling tasks, and a mapper interface, which can affect performance but not correctness, picks processors and memories [26]; Regent gives this a language whose type system checks the privileges and whose compiler turns loops of task launches into index launches [65]. The machine-learning line names the dimensions. Mesh-TensorFlow gives every tensor dimension a name, declares a mesh of processors with named dimensions, and takes a layout that maps tensor names to mesh names; each operation becomes one local operation per processor plus the collectives the layout implies, with an all-reduce over the mesh dimensions of any reduced-out tensor dimension [32].

c) The design space: The three ideas disagree on who decides and when. The polyhedral compilers decide everything at compile time from exact dependences, and can do so because their input is affine. Legion decides at run time from declared privileges, and can do so for any program that names its regions. Mesh-TensorFlow decides at graph construction from a layout the user writes, and its authors argue in an appendix that owner-computes is communication-suboptimal for matrix multiplication, which their layouts avoid by replication [32]. FlexFlow searches the space of per-operator partition configurations with a simulator that profiles each operator once, and its configurations partition output tensors, which is owner-computes [66]; PipeDream partitions layers into stages with a dynamic program over a single-GPU profile and a hierarchical bandwidth model [62].

d) What the era learned: The polyhedral generator reached 32-node speedups of 11.9 to 570 over sequential code across stencils and linear-algebra kernels, and beat hand-written MPI and pghpf on stencils, which its author credits to full space-time tiling [61]. Its stated limitations are that every process holds all data and that communication volume

is not optimal when parts of one flow-out set have different receivers [31]. Legion reached 62.5 times a hand-coded single-GPU version on 96 GPUs for a circuit simulation, with its strongest results depending on application-specific mappers [26]. Regent's authors advise an SPMD structure at the outer level to scale past roughly ten to a hundred nodes, because simultaneous region access is not implemented [65]. FlexFlow found strategies 1.3 to 3.3 times faster than data-parallel and expert strategies with search in seconds to minutes, under the stated assumption that operator times are predictable and data-independent [66].

e) What remained: Each line solved the decision for its own input: affine loops, region-declaring tasks, or operator graphs with static shapes. None of them composed with the others, and the machine-learning line still asked the programmer for the layout. The next era automated the layout.

F. Era 6 (c. 2020–2026): propagation, schedules, and the wafer

a) Machines and the problem: Accelerator pods and GPU clusters with a memory limit per device, and wafer-scale engines with a memory limit per processing element. GSPMD's design is driven by production constraints: one program for all partitions because one program per partition would compile too slowly at thousands of partitions, static shapes because accelerators need them even when dimensions do not divide evenly, and the full operator set [6]. Its measurements use TPUv3 cores with 16 GB each [6]; the Cerebras CS-1 holds 18 GB on the wafer, which bounds the problem size [9].

b) The central idea: Annotate a few tensors with a sharding over a named mesh and let the compiler propagate it. In GSPMD each tensor carries a sharding property, replicated, tiled or partially tiled, and two passes complete the annotations by propagating along preserved dimensions with fixed priorities and then partition every operator into one SPMD program, inserting collectives, halo exchanges and reshardings where shardings disagree; the dense Transformer study annotates seven tensors per layer, about 0.7% of the graph, and there is no cost model [6]. PartIR keeps the model unannotated and makes the strategy a schedule of tactics, each desugaring into tile, atomic and propagate rewrites on a loop-and-slice IR; propagation uses a registry of algebraic equivalences per operator rather than cost heuristics, and the lowering of the core IR to SPMD is proved correct [7]. DISTAL makes the distribution part of a tensor's format and the computation distribution a schedule command: distributing output-indexing index variables gives owner-computes, distributing reduction variables gives distributed reductions, and a rotate command produces systolic patterns [8].

c) The design space: The era's systems differ on what is automated. GSPMD automates completion but not the initial annotations; PartIR automates nothing by default and offers an automatic tactic driven by a simple analytical simulator [7]; Alpa automates everything by an integer program over per-operator algorithms and a dynamic program over stages, with profiling [67]; Kong et al. automate placement and grid shape for tensor DAGs with an SMT solver over a volume cost [41]; DISTAL and the MLIR sparse dialect leave the distribution to the user [8], [68]. The sparse and stencil systems reintroduce the run-time stage and the halo. SpDISTAL derives the partition of a sparse tensor's coordinate tree at run time through image and preimage operations on the Legion runtime, and offers non-zero partitions that balance stored entries instead of coordinates [69]. The shared stencil stack declares halo exchanges in a dmp dialect and lowers them to non-blocking MPI, one exchange per halo [70], and the wafer-scale pipeline reuses the same decomposition to place one column of the domain per processing element and turns the exchanges into chunked prefetches overlapped with computation [71].

d) What the era learned: GSPMD reports 54 to 62% utilisation on 128 to 2048 TPUv3 cores for Transformers up to one trillion parameters, with no external baseline; its pipelining mode supports only homogeneous stages and was 24% slower than pure two-dimensional sharding on the one model where both were tried [6]. PartIR matches GSPMD within one percentage point of utilisation on Transformers, and on U-Net a single PartIR tactic ran out of memory while matching PartIR in GSPMD needed five hand-placed constraints per layer [7]. Alpa matched hand-tuned Megatron-LM on GPT and reached 3.5 to 9.7 times DeepSpeed on mixture-of-experts models, at a compile cost of 2393 seconds for a 39-billion-parameter model on 64 GPUs, mostly profiling [67]. DISTAL's best schedule stayed within 10% of COSMA on 256 nodes with a schedule of about fifteen lines against an estimated five hundred for COSMA's distribution logic [8]. The wafer-scale pipeline slightly outperformed a hand-written kernel on one stencil and supports only star-shaped stencils [71].

e) What remained: The era rebuilt the Era 3 stack for a program class that is kinder to the compiler: a few large operators with static shapes, on homogeneous devices, with memory capacity as the binding constraint. The parts that were hard in 1995 are still hard: irregular ownership is handled only by sparse systems with a run-time stage, redistribution is inserted by propagation but not costed, and no read paper measures productivity with users, although five to seven of the ten Era 6 papers record it as an objective, depending on the coding. Section XIII lists these.

VII. Data Decomposition and Computation Mapping

A. Three classes of ownership

We group the decompositions in the catalog by what the compiler must know to generate code for them (Fig. 4). The first class is closed-form owner functions: block, cyclic, block-cyclic and uneven block. For these the owner and the local index are arithmetic on the global index, and Chatterjee et al.'s finite-state machine generates the local access sequence for cyclic(k) distributions with affine alignment [30]. The second class is declared maps: mapping arrays, graph partitions and user-defined distribution functions. Vienna Fortran's INDIRECT and user-defined distribution functions are of this kind [27], [42], as are CHAOS's partitioner-driven translation tables [51]. For these the owner of an element is data, so

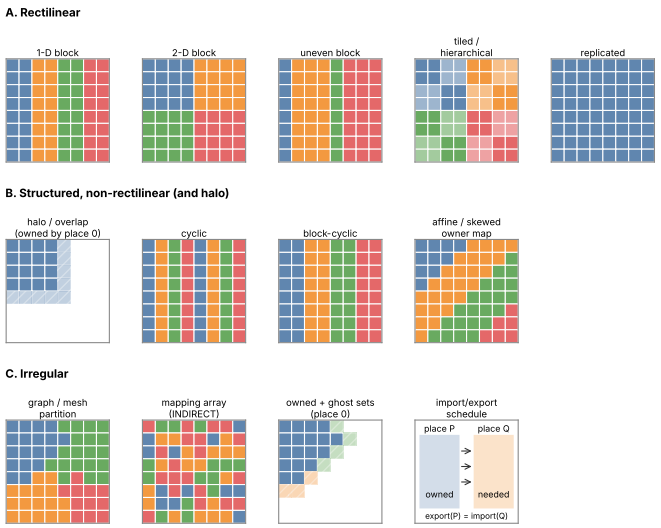


Fig. 4. Data-decomposition classes drawn as example grids coloured by owning place. Rectilinear forms (A) have closed-form owner functions. Structured non-rectilinear forms (B) need block-cyclic address arithmetic. Irregular forms (C) need a mapping array or graph partition and, in the systems surveyed, a run-time stage. Hatched cells are ghost copies. The examples are illustrative.

the compiler needs a table and a run-time stage (Section IX). The third class is propagated shardings: a per-dimension assignment of tensor axes to mesh axes, with replication where no axis is named [6], [32]. Here the compiler needs the mesh and the propagation rules, not a table of owners.

The classes predict the decision procedure. Closed-form owners admit exact analysis, so the compiler can decide statically, as in Koelbel's closed forms [29]. Declared maps force a run-time stage. Propagated shardings admit a static procedure over the graph, but the procedure is heuristic when shardings cannot be inferred uniquely [6].

B. What the decomposition vocabulary shows

The vocabulary pattern in Section XII gives the block-cyclic and mapping-array counts under both codings. Under the first coding halo overlap is recorded in every era (3, 2, 2, 2 and 3 papers in Eras 2 to 6); under the second it is absent from Era 3, so the every-era pattern is suggestive, not established. It appears as overlap areas in Fortran D and Vienna Fortran [42], [43], as halo exchange in the dmp dialect [70], and as neighbour exchange on the wafer-scale fabric [9], [71]. If halo overlap recurs across eras, its persistence suggests that neighbour data movement is a property of stencil-like computation rather than of a language era. Recoding Era 3 is the test.

C. Assigning computation

The computation rules follow the classes. Under closed-form and declared ownership the rule is owner-computes or an equivalent (Kali's on-clauses [28], Fortran D's owner partitioning [43]). Under propagated shardings the rule is that each operator is partitioned from the shardings of its operands [6]. Task-based systems place tasks and regions through a mapper, which decouples the rule from ownership [26]. The owner-rule pattern in Section XII gives the counts under both codings.

D. Alternatives to ownership

Some systems do not decide ownership first. Pipeline systems assign stages of layers to devices and derive the data movement between them [62], and expert parallelism assigns sub-networks to devices [6]. For these the unit of placement is a stage or a sub-network rather than an array element. The catalog records them under computation mapping, and they are the main reason the ownership-based axis does not describe the recent systems completely.

VIII. Communication Generation and Redistribution

Communication is derived in two ways in the catalog. Element-level generation computes, for each reference, which elements move between which places, from the subscripts and the distribution. Collective generation recognises an operator's pattern and emits a collective with participants fixed by the sharding. Which family a system records depends on the coding: the two codings disagree on most systems of Era 4 and Era 5, so we state no trend in the mix (Section XII).

A. Element-level generation

Koelbel derived send, receive and iteration sets in closed form for forall loops over block and cyclic arrays with linear subscripts [29]. Chatterjee et al. generate local addresses and communication sets for cyclic(k) distributions [30]. Shen et al. remove iteration variables that do not change which element or processor is involved, reducing generated communication on one LU kernel [60]. Bondhugula's generator computes flow-out and write-out sets per tile and uses run-time helper functions to find the receiving processes [31], [61]. All four derive communication from affine index expressions. Their generality is limited by the affine assumption, and their cost is paid at compile time or in a small run-time helper. This is the classical regime of the catalog.

B. Collective generation

GSPMD inserts collectives where propagated shardings disagree [6], and the MLIR shard dialect exposes all-gather, all-reduce, all-to-all, reduce-scatter and halo update as operations whose participants follow from the sharding [33]. Kong et al. forbid the crossed mappings that would need point-to-point traffic, so only collectives are generated, and each (operator, tensor) communication vector selects a sub-communicator [41]. Vaidya et al. implement blocking get, set and update-slice operations over MPI all-to-all and all-to-allv [68]. Collective generation trades generality for a simpler model. Once the sharding is fixed the participants are known, so the cost of a collective can be stated in volume alone. Kong et al.'s model is volume-only for this reason, and it is the simplest model in the read set [41].

C. Halo exchange and overlap

Halo exchange is recorded in Eras 2, 3 and 6 under both codings. In Eras 4 and 5 it is recorded only under the first coding, so we do not claim that it recurs in every era (Section VII). Bisbas et al. lower declared halo exchanges to non-blocking sends and receives, and note that each halo is a separate exchange with no overlap and no diagonal exchange [70]. Stawinoga et al. convert halo exchanges into chunked prefetches overlapped with computation on the wafer-scale engine [71]. Rocki et al. overlap receiving with multiplication but keep the all-reduce blocking [9]. Overlap is treated as a property of the halo or the chunk, and reductions are the usual exception. Among the read papers we did not find a reduction overlapped with the computation that feeds it; this is a candidate open problem, to be confirmed by a targeted search.

D. Redistribution

Redistribution is the change of distribution between phases. Ramaswamy et al. represent regular distributions as families of line segments, so the data to move between two processor sets is their intersection [35]. Palermo et al. choose phases by a shortest-path search that includes the redistribution cost [36]. In the recent systems redistribution is a resharding between two shardings, inserted where the propagation finds a mismatch [6], [7]. The concept is the same, the input is different: a pair of line-segment families in the 1990s, a pair of mesh shardings now.

IX. Runtime Analysis and Irregular Computation

The boundary between compiler and runtime is where the systems differ most sharply. the runtime pattern in Section XII states the central regularity under both codings: where ownership is set by an explicit map, the read systems that describe a run-time mechanism for it have a run-time stage, with two named exceptions (Section XII). This section explains the forms that stage takes and the costs they carry.

A. Three forms of run-time stage

The inspector–executor form separates an inspection of the index arrays from the execution that uses its result. Saltz and colleagues' executor runs a table-driven schedule built by a preprocessor, and the executor overhead is measured at about 9–10% for sparse matrix-vector multiplication on an iPSC/2 [34]. Kali caches the results of its inspector across repeated executions [28]. The reuse condition is the decisive design point. Ponnusamy et al. reuse inspector results with a test that is conservative and can add overhead in some cases [51].

The dynamic-scheduling form moves the decision out of the compiler. Legion moves data-movement decisions to a runtime that reasons about logical regions and privileges [26]. Dathathri et al.'s dataflow runtime evaluates task functions per node [64]. The run-time-resolution form is a fallback. Rogers and Pingali combine run-time resolution, which is correct for every reference, with compile-time resolution where ownership can be analysed [39].

B. What the stage costs

The stage costs an inspector pass or a dynamic scheduler on every change of the index arrays. The 1990s systems amortised this over repeated executions of the same loop [28], [34], and the sparse tensor systems do the same over repeated tensor operations [69]. The adaptive variants accept a schedule rebuild when the indirection changes [38]. We did not find a read paper that reports the break-even point against a fully static alternative for the same program, so the trade-off is described but not measured.

C. Where runtime stages are not needed

Three read papers have no run-time stage for their distributions: Gupta and Banerjee's constraint method [40], Lee's dynamic program [13], and Palermo et al.'s phase search [36]. All three decide the distribution at compile time, but over a cost model rather than by directive or analysis (Section V). The compile-time decision is therefore not the same thing as a closed-form one, and the run-time question is separate from the decision-procedure question.

X. Performance Modeling and Optimization Objectives

Three decision procedures appear in the read set: a closed-form or directive decision, an analytic optimisation over a cost model, and a search over configurations scored by profiles or a simulator (Section XII). The procedure determines what the cost model has to represent, and the objectives it optimises.

A. Analytic optimisation in the 1990s

Gupta and Banerjee weight distribution constraints by estimated execution-time penalties, and the authors state that the combined problem is NP-hard and that their estimates rest on compile-time assumptions about loop bounds and branch probabilities [40]. Palermo et al. describe their static pipeline cost model as conservative [36]. Lee's layout choice is a dynamic program over consecutive loops [13]. [TODO: Check how each analytic model in this group represents the network, from the text of [13], [36], [40]; the cards record the assumptions about loop bounds but not yet the network terms.]

B. Volume models in the 2020s

Kong et al. minimise local work plus a weighted communication volume under per-node memory capacity; the cost model is volume-only, with no latency or bandwidth terms, and the paper reports no numeric speedups in its text [41]. GSPMD has no cost model; its sharding is chosen by priority-based propagation [6].

C. Measured and simulated costs from 2019

PipeDream's optimiser uses a single-GPU profile of per-layer compute time, activation size and parameter size, and minimises the time of the slowest stage with a dynamic program over a hierarchical bandwidth model [62]. FlexFlow's

simulator profiles each operator once and re-simulates incrementally, and the authors state that it assumes operator times are predictable and independent of input data [66]. Alpa uses profiled stage costs with an integer program for intra-operator choices and dynamic programming for stages [67], and PartIR's automatic tactic uses an analytical simulator with Monte Carlo tree search [7].

D. What the objective shift implies

Section XII gives the counts. Taken together, the analytic group models the network and the search group models the program's execution. An analytic model needs a network cost accurate enough to rank choices; a profile or simulator needs only the cost of each operator on the target, which can be measured or simulated without a network model [62], [66]. This is the inference behind the shift, and it is testable: a system that uses a network model and a profile on the same workload, compared on the same machine, would separate the two explanations. We did not find a read paper that makes this comparison.

E. Which objectives can be reported

The objectives axis is coded unreliably: the two codings agree exactly on 3 of 54 papers. We therefore use only two objective patterns, and both hold under both codings. Message count is the most frequent recorded objective of Era 3, and it is nearly absent from Era 6 (Section XII). Search cost is recorded only in Eras 5 and 6. The claims about memory fit and productivity that the first coding suggested are withdrawn, because the second coding does not show them. Productivity is recorded as an objective for five to seven of the ten Era 6 papers, but no read paper measures it with users, so the objective is asserted, not evaluated.

XI. Multi-GPU, ML, Spatial, and AI-Accelerator Compilers

The recent systems share one design choice: the programmer writes an unpartitioned model and a mesh, and the compiler propagates a layout through the program. The choice is the source of the propagation value on the owner-rule axis (Section XII), and the rest of this section examines what it changes and what it leaves to the programmer.

A. Propagation of shardings

Mesh-TensorFlow expresses a program as named tensor dimensions mapped onto a mesh of processors, so any dimension can be split and the collectives follow from the layouts [32]. GSPMD completes sharding annotations by priority-based propagation and partitions every operator, covering data, model, spatial and pipeline parallelism through one mechanism [6]. PartIR separates partitioning tactics from the model and composes them on an MLIR-based IR, and its propagation uses rewrite rules rather than cost heuristics [7]. Its authors report 50–62% TPUv3 utilisation on up to 2048 cores for GSPMD, without a cost model or a search [6].

Propagation moves the decision from the statements to the values. That makes the compiler's job a constraint problem over the graph, which is heuristic when the constraints are not unique. GSPMD's priority scheme and PartIR's tactic order are two answers to the same ambiguity, and PartIR's authors note that propagation can be blocked by reshapes and non-divisible shapes [7].

B. Search over parallelism plans

FlexFlow searches per-operator configurations with a simulator [66], and Alpa searches intra-operator shardings with an integer program and inter-operator stages with dynamic programming [67]. These systems automate the choice that the propagation systems leave to the programmer, the mesh shape and the stage boundaries. Alpa's authors state that cross-stage communication is not modelled [67], which is the same gap that the analytic group of Section X had for the network.

C. Pipeline and training systems

PipeDream partitions layers into stages from a profile and schedules them with weight stashing [62]; GPipe pipelines micro-batches [72]; Megatron-LM implements tensor parallelism by hand [73]; and ZeRO is catalogued for model-state sharding [74], whose card is pending. We classify the last four as bridge entries because their plan generation is manual or limited.

D. Tensor and sparse-tensor compilers

DISTAL and SpDISTAL bring distribution formats and scheduling commands to dense and sparse tensor algebra, compiled to Legion [8], [69]. Kong et al. derive mappings for tensor DAGs with an SMT solver [41]. Vaidya et al. express distributed sparse tensors as an MLIR dialect that lowers to MPI, with partitions fixed by the programmer [68]. The sparse systems are the only read papers with a sparse-tensor decomposition, and each keeps the partition choice with the user or with a run-time inspector.

E. Spatial and wafer-scale targets

Rocki et al. hand-map a BiCGStab solver onto the Cerebras engine with one mesh column per core and a hardware-routed AllReduce [9]. Stawinoga et al. compile stencil DSLs to the Cerebras software language through three new dialects, with a runtime library that supports only star-shaped stencils [71]. Jacquelin et al. study scalable distributed high-order stencil computations [75]; the machine and results are pending a read of the paper. Both read wafer-scale papers keep the decomposition static and place one tile per processing element. The compiler's contribution is the communication schedule and the mapping of loops to the fabric, not the choice of decomposition.

[TODO: Add a card for Tiramisu [76], which the catalog lists with distributed-memory support, and for the 2024 Shardy and MLIR documentation once read in full. The accelerator-vendor compilers named in the brief (Graphcore, SambaNova, Tenstorrent, Groq) are not yet catalogued.]

XII. Cross-Era Synthesis

Section VI told each era on its own terms. This section asks what repeats. It has four parts: the mechanism-level correspondences between the 1990s and the 2020s, with the evidence on both sides; the counted patterns across the 54 read papers; the four causes of HPF's decline applied to today's sharding compilers; and the generalizations we are prepared to state, each with its evidence strength and the observation that would refute it.

A. Then and now: the same mechanisms under new names

Table I shows that the correspondences are not loose analogies: seven of the eight mechanisms have the same structure on both sides. The citation column shows how uneven the memory is. Only the separation of mapping from program is cited by every newer system, through Sequoia, Legion and DISTAL to PartIR. For alignment, owner-computes and layout-derived communication, some newer papers cite the older line (DISTAL and Kong et al. do; Mesh-TensorFlow and GSPMD do not). For the processor arrangement, redistribution and the inspector, none of the newer papers' cards records a citation to the 1990s work. The clearest single instance is DISTAL's scheduling rule: distributing an output-indexing variable gives owner-computes, distributing a reduction variable gives a distributed reduction. That is Fortran D's local-iteration-set rule and its relaxation for reductions, stated in 2022 as two commands in a schedule.

B. What changed between the two sides

Three differences recur in the right-hand column. First, the unit of reasoning moved from the array element and the loop statement to the tensor dimension and the operator. Second, the arrangement of places became the unit of cost: a mesh axis names a collective's participants and a bandwidth, where a processor arrangement named only positions. Third, the decision is taken by propagation over values rather than by a rule over statements. The program class made all three possible: a graph of a few large operators with static shapes admits per-operator rewrite rules, and homogeneous devices admit a cost in bytes per mesh axis. The 1990s compilers faced loops with arbitrary affine subscripts over machines with a startup cost two hundred times the byte cost [45], and had to count messages.

C. Patterns tested on the read set

Each of the 54 read papers was coded twice on five axes: data decomposition, computation mapping, communication, the compiler/runtime boundary and objectives. The first coding is the catalog's. The second was made by an independent coder who had not seen the first and worked only from the card text. Exact-set agreement is 26% for data decomposition, 20% for computation mapping, 20% for communication, 52% for the compiler/runtime boundary and 6% for objectives. A pattern is reported here only when its direction holds under both codings and their intersection. Table II gives the counts, and the repository gives the full agreement tables.

a) Search-based decomposition appears only from 2019: Papers that choose a decomposition by profile or search are recorded in Era 5 (two papers under both codings) and Era 6 (one or two), and in none of Eras 2 to 4 under the second coding. The first coding places one such paper in Era 2, Fortran D's evaluation, whose card states that the training-set cost estimate is planned future work [45]. We treat it as an exception. Search and profiling enter the decomposition only with the systems of 2019.

b) Message count led the 1990s and is nearly absent later: Message count is the most frequent recorded objective of Era 3 under both codings, with 11 and 9 papers. The margin is narrow: the next most frequent objective has 10 and 8. It falls to at most one paper in Era 6 under either coding. The 1990s evaluations counted messages because, on the machines they used, startup cost dominated (Section VI). The objective counts for the other eras are not stable enough to say more.

c) Explicit maps and their run-time stage: Where ownership is set by an explicit map (a mapping array, a graph partition or a sparse partition), the read papers that describe a run-time mechanism for resolving it have a run-time stage: 12 of 14 under the first coding and 11 of 12 under the second. The exceptions have specific causes. Vienna Fortran's report says that its INDIRECT references need run-time techniques from Kali and PARTI but describes none [27], [42]. The second coder tagged Chapel's planned graph-partitioned distribution without a run-time stage [57]. The pattern is suggestive rather than settled. The exceptions show that the boundary between a planned distribution and a data-dependent one is hard to code from text.

d) Block-cyclic and mapping arrays stop after Era 4: Block-cyclic decompositions appear in 9 and 7 Era 3 papers under the two codings, in 3 Era 4 papers under both, and in none of the Era 5 or Era 6 papers. Mapping arrays follow the same course: 4 and 3 in Era 3, one in Era 4, and none later. This is a claim about the read set, not about the field. The Era 4 PGAS and HPCS papers are the least covered, and the pattern should be tested on them first.

e) Model-state sharding and sparse partitions are recent: Model-state sharding is recorded from Era 5 under both codings and in none of the earlier eras. Sparse-tensor partitions appear only in Era 6, in one or two papers.

f) Partition propagation appears only in Era 6, and owner-computes persists: Partition propagation is recorded for two papers in Era 6 under both codings and for none earlier. Owner-computes is recorded in every era under both codings, with six or more papers in Eras 2 and 3 and five or six of the ten Era 6 papers. The rule is not replaced in the read set. Propagation is a second mechanism for the same question in two systems.

g) What the codings withdraw: Four claims of the first analysis are withdrawn. Element-level communication is recorded for 8 of 9 Era 4 papers under the second coding, so it does not disappear after the 2000s. Productivity is recorded for 8 of 11 Era 2 papers under the second coding, so it does not rise from none to seven. Local memory fit is recorded for at most two papers in any era under the second coding, so it does not grow to eight. Halo overlap is recorded in every era

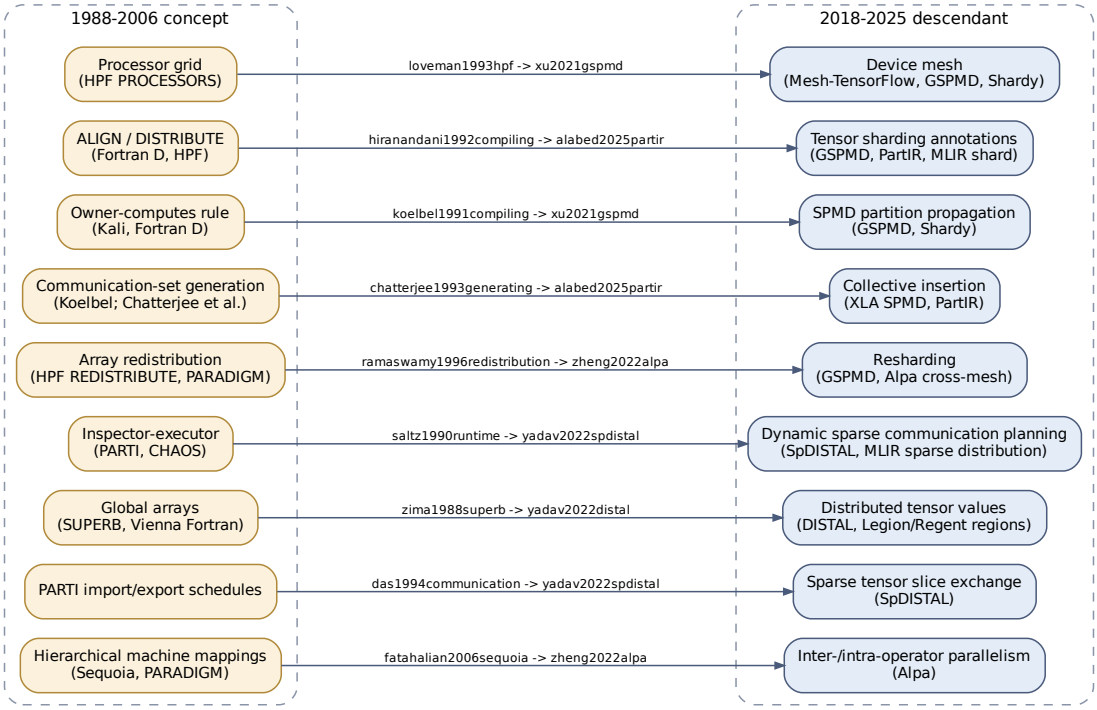


Fig. 5. Concepts of 1988–2006 and their 2018–2025 descendants. Each edge names the catalog entries that document the older and the newer form. The pairing asserts functional correspondence, demonstrated in Table I; it does not assert that the newer work cites the older one, and in several pairs it does not.

under the first coding but not in Era 3 under the second. Halo exchange is recorded in Eras 2, 3 and 6 under both codings and in Eras 4 and 5 only under the first.

D. HPF's four causes, applied to today

Kennedy, Koelbel and Zima attribute HPF's decline to immature compilers combined with an impatient community, insufficient support for important features, inconsistent implementations, and the complex link between program and performance [48]. Each cause can be checked against the sharding compilers from their own cards.

- *Compiler maturity.* Different from HPF. GSPMD's paper reports it already in use for LaMDA and other production models [6]; HPF's compilers followed its standard. The sharding compilers were built inside the frameworks they serve, which removes the gap that HPF's vendors had to close.
- *Feature support.* Same risk, different features. HPF omitted irregular and user-defined distributions [4]. GSPMD's pipelining supports only homogeneous stages [6]; PartIR's propagation is blocked by reshapes and non-divisible shapes and MPMD is future work [7]; Alpa handles only static graphs on homogeneous nodes [67]; and sparse tensors are served by separate systems [68], [69].
- *Inconsistent implementations.* Present. GSPMD, PartIR and Alpa take different inputs and make different decisions on the same model; PartIR's authors state that matching its U-Net result in GSPMD took five hand-placed constraints per layer [7]. The OpenXLA project describes Shardy as combining the two [77], which is the consolidation HPF never had.
- *Program-to-performance link.* The cause the new designs address most directly. PartIR is built around predictability: a schedule whose effect the user can inspect per tactic, and propagation without cost heuristics, because its authors judge automatic partitioners unpredictable [7]. GSPMD's own authors note that reported efficiency depends on the initial annotations, and the three attempts in their Table 1 show it [6]. Alpa's 2393-second compile for one model is the search-cost form of the same link [67].

On this evidence the sharding compilers have removed the first cause, share the second and third, and have made the fourth a design goal rather than a side effect. Whether that is enough is the question the next decade answers; the catalog records what would count as evidence.

E. Generalizations

Table III states what we are prepared to claim. Strength is "supported across codings" when the pattern holds under both codings and their intersection, "suggested" when one coding or a small sample qualifies it, and the refuting observation is the one that would overturn it.

TABLE I
THEN AND NOW. EACH ROW PAIRS A MECHANISM OF THE 1988–2000 SYSTEMS WITH ITS FORM IN THE 2018–2025 SYSTEMS, WITH THE EVIDENCE FROM THE PAPER CARDS AND THE RESPECT IN WHICH THE NEW FORM DIFFERS. THE LAST COLUMN RECORDS WHETHER THE NEWER PAPERS' TEXTS CITE THE OLDER LINE OF WORK, PER THEIR CARDS: YES FOR ALL OF THEM, PARTLY FOR SOME, NO FOR NONE, UNCHECKED WHERE THE CARDS DO NOT SAY.

Mechanism	Then (1988–2000)	Now (2018–2025)	What changed	Cites older line?
Processor arrangement	HPF PROCESSORS: a rectilinear arrangement onto which a template is distributed per dimension [4]; Kali's processor array with a tilde-range resolved at run time [28].	Mesh-TensorFlow's processor mesh with named dimensions [32]; GSPMD's device mesh whose order is preserved in collectives so users can match the topology [6]; Alpa's $N \times M$ mesh with restricted submesh shapes [67].	The arrangement is now also the unit of collectives and of cost; HPF's was only a naming of places.	no
Alignment and distribution	Fortran D aligns arrays to a decomposition, then distributes the decomposition; HPF's TEMPLATE occupies no storage and arrays align to it with offsets, collapse and replication [4], [43].	Mesh-TensorFlow's layout is a partial map from tensor-dimension names to mesh-dimension names, and dimensions with no mapping are replicated [32]; DISTAL's tensor distribution notation names tensor and machine dimensions, with fix and broadcast for unmatched machine dimensions [8].	Alignment is by shared dimension name rather than by an explicit template; replication is the default for unmatched dimensions in both.	partly
Owner-computes	Kali's doall on owner(A(i)) [28]; Fortran D's local iteration set from the left-hand side [43]; relaxed by DEC's compiler, which computes where operands are aligned when that moves less data [47], and by CHAOS's "almost owner computes" [51].	DISTAL: distributing output-indexing variables gives owner-computes, distributing reduction variables gives distributed reductions [8]; FlexFlow partitions output tensors, which is owner-computes [66]; Mesh-TensorFlow argues owner-computes is communication-suboptimal for matrix multiply and uses replication instead [32]; GSPMD and PartIR replace the rule with propagation [6], [7].	The 1995 relaxation and the 2018 critique make the same point; the new systems make relaxation the default by propagating values rather than placing statements.	partly
Communication from the layout	Kali's in and out sets per loop [28]; Koelbel's closed-form send and receive sets [29]; DEC's split of expressions into communication-free regions with transfers materialised as assignments to temporaries [47].	Mesh-TensorFlow: an all-reduce over the mesh dimensions of reduced-out tensor dimensions, an all-to-all when two dimensions share a mesh dimension [32]; GSPMD inserts collectives where shardings disagree [6]; Kong et al. forbid mappings that would need point-to-point traffic [41].	The unit of communication is the operator and the collective, not the element and the message; this is what makes a volume-only cost model sufficient.	partly
Redistribution	HPF REDISTRIBUTE; Vienna Fortran's DISTRIBUTE statement with NOTRANSFER [27]; Palermo et al.'s phase search charging for redistribution [36]; Ramaswamy et al.'s line-segment intersections [35].	GSPMD's resharding composed from all-gather, all-to-all and collective-permute [6]; Alpa's tabulated resharding costs between sharding specs, and cross-mesh resharding as many-to-many multicast [67]; PartIR's redistribute lowered to one all-gather and one all-slice [7].	The 1996 work optimised the data movement of one redistribution; the 2022 work prices redistributions inside a global search and does not optimise each one.	no
Inspector and executor	PARTI's preprocessor and schedule executer [34]; CHAOS's reuse of inspector results under a modification counter [51]; Kali's cached inspector [28].	SpDISTAL's derivation of sparse partitions at run time through image and preimage operations on Legion [69]; PartTensor's get-slice requests exchanged by all-to-all at run time [68]; Titanium's inspector-executor for a[b[i]] [56].	The schedule is now a runtime partition object rather than a message list, and reuse is the runtime's coherence rather than a counter.	no
Halo and overlap	SUPERB and Vienna Fortran overlap areas [10], [42]; Fortran D overlaps for contiguous nonlocal stencil data [45]; DEC's shadow edges [47].	GSPMD's halo exchange via collective-permute with maximum halo then trimming [6]; the dmp dialect's declared halo swaps [70]; chunked prefetches on the wafer [71].	Unchanged in concept; the only mechanism in the catalog present in every era.	unchecked
Separation of mapping from program	Sequoia's per-machine mapping specification [53]; Chapel's distribution classes [57]; Co-Array Fortran's stated separation of work from data distribution [54].	DISTAL's schedule separate from the tensor expression [8]; PartIR's schedule of tactics separate from the model [7]; Legion's mapper that cannot affect correctness [26].	The separation is the same design; PartIR's authors cite DISTAL and Halide for it, and Legion's authors cite Sequoia.	yes

F. What the generalizations say together

G1 and G2 are two answers to one limit. A compiler cannot decide the owner of data it does not know until run time, so it adds a run-time stage (G1). A compiler that must choose among decompositions without a program-level analysis searches configurations by measurement (G2). G4 and G5 describe which decompositions the two answers kept: the cyclic families of the 1990s give way to block shardings, mesh-named layouts and sparse partitions. G3 is the one objective pattern that survives both codings, and we infer that it is the reason the 1990s systems minimised messages.

G. What the generalizations do not say

The read set has no Era 0 or Era 1 paper, so none of this is a claim about the foundations. The counts are over 54 read papers, and 51 further catalog entries rest on bibliographic records alone. The two codings agree exactly on only a minority of papers on every axis, and the objectives axis is not coded reliably; patterns that depend only on it are not reported. The then-and-now rows are structural comparisons by one reader of two cards each, pending a second reader on the citation column.

[TODO: Test G2 and G6 on the Era 4 PGAS and HPCS papers, which are the least covered era in the read set.]

XIII. OPEN PROBLEMS

Each problem below follows from a generalization in Table III and names the evidence that would close it.

- **Measured versus modelled cost (G2).** No read paper compares a network model against a profile on one workload. Until one does, the shift from analytic to measured optimisation is an explanation, not a result [40], [66].
- **Unified ownership and layout IR (G1, G7).** Closed-form owners, declared maps and propagated shardings need

TABLE II
COUNTS OF READ PAPERS PER ERA UNDER THE FIRST CODING AND THE INDEPENDENT SECOND CODING, WRITTEN FIRST/SECOND. ERA SAMPLE SIZES ARE 11, 15, 9, 9 AND 10. A MEASURE WHOSE TWO COUNTS DISAGREE IN DIRECTION IS NOT REPORTED AS A FINDING.

Measure	Era 2	Era 3	Era 4	Era 5	Era 6
search-based decomposition	1/0	0/0	0/0	2/2	2/1
analytic optimisation	1/2	2/3	0/1	0/0	1/2
objective: message count	4/5	11/9	4/0	3/1	1/0
objective: productivity	0/8	2/8	4/4	5/2	7/5
block-cyclic	6/5	9/7	3/3	0/0	0/0
mapping array	5/6	4/3	1/1	0/0	0/0
model-state sharding	0/0	0/0	0/0	1/3	3/3
sparse-tensor partition	0/0	0/0	0/0	0/0	2/1
halo overlap (decomposition)	3/1	2/0	2/1	2/2	3/2
owner-computes	6/6	11/8	3/3	2/2	6/5
partition propagation	0/0	0/0	0/0	0/0	2/2
element-level communication only	4/4	4/6	2/8	1/5	0/2
halo exchange (communication)	3/1	3/3	3/0	1/0	4/4

TABLE III
GENERALIZATIONS, THEIR EVIDENCE STRENGTH IN THE READ SET, THE OBSERVATION THAT WOULD REFUTE EACH, AND THE TEST THAT WOULD SETTLE IT.

ID	Generalization	Strength	Refuting observation	Test
G1	Where ownership is set by an explicit map, the read systems that describe a run-time mechanism for it have a run-time stage.	suggested (12 of 14; 11 of 12)	An explicit-map read paper that describes a mechanism and has no run-time stage.	Read the Vienna Fortran VFCS implementation; resolve the Chapel coding.
G2	Choosing a decomposition by profile or search appears only from 2019.	supported across codings	A read system before 2019 that chooses a decomposition from profiles or search.	Read the 2010–2018 systems in the catalog that profile.
G3	Message count is the leading recorded objective of Era 3 and is nearly absent from Era 6.	supported across codings (narrow margin)	A read Era 5 or Era 6 system that records message count as its leading objective.	Recode objectives with a codebook; current agreement is 6%.
G4	Block-cyclic and mapping-array decompositions do not appear in read papers from 2010 onward.	supported across codings	A read paper from 2010 onward with block-cyclic or mapping-array decomposition.	Read the 2010–2019 HPF follow-ups and the Era 4 PGAS and HPCS papers.
G5	Model-state sharding is recorded from 2010; sparse-tensor partitions only from 2020.	supported across codings (small samples)	A read paper before 2010 with model-state sharding, or before 2020 with sparse-tensor partitions.	Read ZeRO and the 2010s sparse systems.
G6	Partition propagation appears only in Era 6, and owner-computes persists alongside it.	supported across codings (two systems)	A propagation system before 2020, or an Era 6 read set with no owner-computes under both codings.	Read Shardy and the MLIR shard dialect in full; read the remaining 2020s systems.
G7	Halo overlap recurs across the eras.	suggested (absent from Era 3 under the second coding)	A recoding that finds halo overlap in Era 3, or a read era with none.	Recode the Era 3 papers for halo overlap.
G8	Seven of the eight 1990s mechanisms in Table I reappear with the same structure in the 2020s systems.	suggested (structural, not counted)	A structural difference missed in the table.	A second reader checks each row against both cards and reference lists.

different analyses. The MLIR shard dialect is the nearest candidate for a shared representation, and the sparse dialect of Vaidya et al. is a second [33], [68].

- **Propagation under constraints.** PartIR's authors report that propagation is blocked by reshapes and non-divisible shapes [7]; GSPMD's sharding completion is heuristic [6]. The conditions under which propagation is unique are not characterised.
- **Overlap of reductions.** Halo exchanges overlap with computation, but among the read papers we did not find a reduction overlapped with the computation that feeds it [9], [70].
- **Break-even of run-time stages.** No read paper reports when an inspector pays for itself against a fully static schedule [34], [51].
- **Correctness of generated communication.** Verification is rarely addressed. The PartIR core is the exception, with a lowering that its authors report as proved [7].
- **Wafer-scale and fine-grained memory.** Both read wafer-scale compilers keep the decomposition static and one tile per processing element, and the stencil runtime supports only star-shaped patterns [9], [71].
- **Productivity.** Productivity is recorded as an objective for five to seven of the ten Era 6 papers, depending on the coding, but none of the read papers measures it with users [6], [7].

[TODO: Each item needs its test from Table III run, or an explicit statement that it was not run.]

XIV. CONCLUSION

Compilers for distributed memory have answered the same three questions in every era: who owns each element, who computes each operation, and what communication connects them. In the read set, the answers that survive two independent codings are narrow. Where ownership is set by an explicit map, the systems that describe a run-time mechanism for it have a run-time stage. Decompositions chosen by profile or search appear only from 2019. Message count led the objectives of the 1990s and is nearly absent later. Block-cyclic and mapping-array decompositions do not appear after 2010. Partition propagation appears only in the 2020s, beside owner-computes. We state these as generalizations with their evidence and their refuting observations (Table III). Several patterns that the first coding suggested did not survive the second, and

we report them as withdrawn. The taxonomy, the catalog, the comparison tables and the sensitivity analysis are generated from the same records, so each count can be regenerated and challenged. [TODO: Finalise after the second reader on the then-and-now table and the Era 4 test.]

REFERENCES

[1] D. Callahan and K. Kennedy, "Compiling programs for distributed-memory multiprocessors," *The Journal of Supercomputing*, 1988.

[2] H. P. Zima, H.-J. Bast, and M. Gerndt, "Superb: A tool for semi-automatic mimd/simd parallelization," *Parallel Computing*, 1988.

[3] M. Chen, Y.-I. Choo, and J. Li, "Compiling parallel programs by optimizing performance," *The Journal of Supercomputing*, 1988.

[4] D. Loveman, "High performance fortran," *IEEE Parallel & Distributed Technology: Systems & Applications*, 1993.

[5] W. Gropp, E. Lusk, N. Doss, and A. Skjellum, "A high-performance, portable implementation of the mpi message passing interface standard," *Parallel Computing*, 1996.

[6] Y. Xu, H. Lee, D. Chen, B. Hechtman, Y. Huang, R. Joshi, M. Krikun, D. Lepikhin, A. Ly, M. Maggioni, R. Pang, N. Shazeer, S. Wang, T. Wang, Y. Wu, and Z. Chen, "Gspmd: General and scalable parallelization for ml computation graphs," arXiv preprint, 2021. [Online]. Available: https://arxiv.org/pdf/2105.04663

[7] S. Alabed, D. Belov, B. Chrzaszcz, J. Franco, D. Grewe, D. Maclaurin, J. Molloy, T. Natan, T. Norman, X. Pan, A. Paszke, N. A. Rink, M. Schaarschmidt, T. Sitdikov, A. Swietlik, D. Vytiniotis, and J. Wee, "Partir: Composing spmd partitioning strategies for machine learning," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2025.

[8] R. Yadav, A. Aiken, and F. Kjolstad, "Distal: the distributed tensor algebra compiler," in *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, 2022.

[9] K. Rocki, D. V. Essendelft, I. Sharapov, R. Schreiber, M. Morrison, V. Kibardin, A. Portnoy, J. F. Dietiker, M. Syamlal, and M. James, "Fast stencil-code computation on a wafer-scale processor," in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2020.

[10] H. Zima and B. Chapman, "Compiling for distributed-memory systems," *Proceedings of the IEEE*, 1993.

[11] S. Benkner and H. Zima, "Compiling high performance fortran for distributed-memory architectures," *Parallel Computing*, 1999.

[12] D. Unat, A. Dubey, T. Hoefler, J. Shalf, M. Abraham, M. Bianco, B. L. Chamberlain, R. Cledat, H. C. Edwards, H. Finkel, K. Fuerlinger, F. Hannig, E. Jeannot, A. Kamil, J. Keasler, P. H. J. Kelly, V. Leung, H. Ltaief, N. Maruyama, C. J. Newburn, and M. Pericas, "Trends in data locality abstractions for hpc systems," *IEEE Transactions on Parallel and Distributed Systems*, 2017.

[13] P. Lee, "Techniques for compiling programs on distributed memory multicomputers," *Parallel Computing*, 1995.

[14] J. W. Backus, H. Stern, I. Ziller, R. A. Hughes, R. Nutt, R. J. Beeber, S. Best, R. Goldberg, L. M. Haibt, H. L. Herrick, R. A. Nelson, D. Sayre, and P. B. Sheridan, "The fortran automatic coding system," *Papers presented at the February 26-28, 1957, western joint computer conference: Techniques for reliability on - IRE-AIEE-ACM '57 (Western)*, 1957.

[15] L. Lamport, "The parallel execution of do loops," *Communications of the ACM*, 1974.

[16] D. J. Kuck, R. H. Kuhn, D. A. Padua, B. Leasure, and M. Wolfe, "Dependence graphs and compiler optimizations," in *Proceedings of the 8th ACM SIGPLAN-SIGACT symposium on Principles of programming languages - POPL '81*, 1981.

[17] R. Allen and K. Kennedy, "Automatic translation of fortran programs to vector form," *ACM Transactions on Programming Languages and Systems*, 1987.

[18] G. Barnes, R. Brown, M. Kato, D. Kuck, D. Slotnick, and R. Stokes, "The illiac iv computer," *IEEE Transactions on Computers*, 1968.

[19] R. M. Russell, "The cray-1 computer system," *Communications of the ACM*, 1978.

[20] J. B. Dennis and D. P. Misunas, "A preliminary architecture for a basic data-flow processor," *Proceedings of the 2nd annual symposium on Computer architecture - ISCA '75*, 1975.

[21] D. P. Misunas, "Report on the workshop on data flow computer and program organization," MIT Laboratory for Computer Science, Technical Memo MIT/LCS/TM-92, Tech. Rep., 1977, supplied copy; no public record located.

[22] Kung, "Why systolic architectures?" *Computer*, 1982.

[23] C. L. Seitz, "The cosmic cube," *Communications of the ACM*, 1985.

[24] W. D. Hillis and G. L. Steele, "Data parallel algorithms," *Communications of the ACM*, 1986.

[25] H. P. F. Forum, "High performance fortran language specification," *ACM SIGPLAN Fortran Forum*, 1993.

[26] M. Bauer, S. Treichler, E. Slaughter, and A. Aiken, "Legion: Expressing locality and independence with logical regions," in *2012 International Conference for High Performance Computing, Networking, Storage and Analysis*, 2012.

[27] B. Chapman, P. Mehrotra, and H. Zima, "Vienna fortran: A fortran language extension for distributed memory multiprocessors," NASA Contractor Report 187634 / ICASE Report 91-72, Tech. Rep., 1991. [Online]. Available: https://ntrs.nasa.gov/citations/19920002452

[28] P. Mehrotra and J. V. Rosendale, "Programming distributed memory architectures using kali," NASA Contractor Report 187453 / ICASE Report 90-69, Tech. Rep., 1990. [Online]. Available: https://ntrs.nasa.gov/citations/19910002904

[29] C. Koelbel, "Compile-time generation of regular communications patterns," in *Proceedings of the 1991 ACM/IEEE conference on Supercomputing - Supercomputing '91*, 1991.

[30] S. Chatterjee, J. R. Gilbert, F. J. E. Long, R. Schreiber, and S.-H. Teng, "Generating local addresses and communication sets for data-parallel programs," in *Proceedings of the fourth ACM SIGPLAN symposium on Principles and practice of parallel programming - PPOPP '93*, 1993.

[31] U. Bondhugula, "Compiling affine loop nests for distributed-memory parallel architectures," in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, 2013.

[32] N. Shazeer, Y. Cheng, N. Parmar, D. Tran, A. Vaswani, P. Koanantakool, P. Hawkins, H. Lee, M. Hong, C. Young, R. Sepassi, and B. Hechtman, "Mesh-tensorflow: Deep learning for supercomputers," arXiv preprint (also NeurIPS 2018), 2018. [Online]. Available: https://arxiv.org/pdf/1811.02084

[33] L. community, "Mlir 'shard' dialect (official documentation; formerly 'mesh')," MLIR documentation (mlir.llvm.org), 2025. [Online]. Available: https://mlir.llvm.org/docs/Dialects/Shard/

[34] J. Saltz, K. Crowley, R. Michandaney, and H. Berryman, "Run-time scheduling and execution of loops on message passing machines," *Journal of Parallel and Distributed Computing*, 1990.

[35] S. Ramaswamy, B. Simons, and P. Banerjee, "Optimizations for efficient array redistribution on distributed memory multicomputers," *Journal of Parallel and Distributed Computing*, 1996.

[36] D. J. Palermo, E. W. H. IV, and P. Banerjee, "Dynamic data partitioning for distributed-memory multicomputers," *Journal of Parallel and Distributed Computing*, 1996.

[37] R. Das, R. Ponnusamy, J. Saltz, and D. Mavriplis, "Distributed memory compiler methods for irregular problems: data copy reuse and runtime partitioning," NASA Contractor Report 187635 / ICASE Report 91-73, Tech. Rep., 1991. [Online]. Available: https://ntrs.nasa.gov/citations/19920002451

[38] Y. Hwang, B. Moon, S. D. Sharma, R. Ponnusamy, R. Das, and J. H. Saltz, "Runtime and language support for compiling adaptive irregular programs on distributed-memory machines," *Software: Practice and Experience*, 1995.

[39] A. Rogers and K. Pingali, "Compiling for distributed memory architectures," *IEEE Transactions on Parallel and Distributed Systems*, 1994.

[40] M. Gupta and P. Banerjee, "Demonstration of automatic data partitioning techniques for parallelizing compilers on multicomputers," *IEEE Transactions on Parallel and Distributed Systems*, 1992.

[41] M. Kong, R. A. Yosef, A. Rountev, and P. Sadayappan, "Automatic generation of distributed-memory mappings for tensor computations," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2023.

[42] B. Chapman, P. Mehrotra, and H. Zima, "Programming in vienna fortran," *Scientific Programming*, 1992.

[43] S. Hiranandani, K. Kennedy, and C.-W. Tseng, "Compiler optimizations for fortran d on mimd distributed-memory machines," in *Proceedings of the 1991 ACM/IEEE conference on Supercomputing - Supercomputing '91*, 1991.

[44] J. Saltz, H. Berryman, and J. Wu, "Multiprocessors and run-time compilation," *Concurrency: Practice and Experience*, 1991.

[45] S. Hiranandani, K. Kennedy, and C.-W. Tseng, "Evaluation of compiler optimizations for fortran d on mimd distributed memory machines," in *Proceedings of the 6th international conference on Supercomputing - ICS '92*, 1992.

[46] P. Banerjee, J. Chandy, M. Gupta, E. Hodges, J. Holm, A. Lain, D. Palermo, S. Ramaswamy, and E. Su, "The paradigm compiler for distributed-memory multicomputers," *Computer*, 1995.

[47] J. Harris, J. A. Bircsak, M. R. Bolduc, J. A. Diewald, I. Gale, N. W. Johnson, S. Lee, C. A. Nelson, and C. D. Offner, "Compiling high performance fortran for distributed-memory systems," *Digital Technical Journal*, 1995. [Online]. Available: https://ftp.math.utah.edu/pub/tex/bib/idx/dectechj/7/3/5_23.html

[48] K. Kennedy, C. Koelbel, and H. Zima, "The rise and fall of high performance fortran," in *Proceedings of the third ACM SIGPLAN conference on History of programming languages*, 2007.

[49] E. Su, A. Lain, S. Ramaswamy, D. J. Palermo, E. W. H. IV, and P. Banerjee, "Advanced compilation techniques in the paradigm compiler for distributed-memory multicomputers," in *Proceedings of the 9th International Conference on Supercomputing (ICS '95)*, 1995.

[50] M. Gupta, S. Midkiff, E. Schonberg, V. Seshadri, D. Shields, K.-Y. Wang, W.-M. Ching, and T. Ngo, "An hpf compiler for the ibm sp2," in *Proceedings of the 1995 ACM/IEEE conference on Supercomputing (CDROM) - Supercomputing '95*, 1995.

[51] R. Ponnusamy, J. Saltz, A. Choudhary, Y.-S. Hwang, and G. Fox, "Runtime support and compilation methods for user-specified irregular data distributions," *IEEE Transactions on Parallel and Distributed Systems*, 1995.

[52] R. Chandra, D.-K. Chen, R. Cox, D. E. Maydan, N. Nedeljkovic, and J. M. Anderson, "Data distribution support on distributed shared memory multiprocessors," in *Proceedings of the ACM SIGPLAN 1997 Conference on Programming Language Design and Implementation (PLDI '97)*, 1997.

[53] K. Fatahalian, T. Knight, M. Houston, M. Erez, D. Horn, L. Leem, J. Park, M. Ren, A. Aiken, W. Dally, and P. Hanrahan, "Sequoia: Programming the memory hierarchy," in *ACM/IEEE SC 2006 Conference (SC'06)*, 2006.

[54] R. W. Numrich and J. Reid, "Co-array fortran for parallel programming," *ACM SIGPLAN Fortran Forum*, 1998.

[55] W.-Y. Chen, D. Bonachea, J. Duell, P. Husbands, C. Iancu, and K. Yelick, "A performance analysis of the berkeley upc compiler," in *Proceedings of the 17th annual international conference on Supercomputing*, 2003.

[56] K. Yelick, P. Hilfinger, S. Graham, D. Bonachea, and J. Su, "Parallel languages and compilers: Perspective from the titanium experience," *The International Journal of High Performance Computing Applications*, vol. 21, no. 3, 2007.

[57] B. Chamberlain, D. Callahan, and H. Zima, "Parallel programmability and the chapel language," *The International Journal of High Performance Computing Applications*, 2007.

[58] P. Tang and J. Xue, "Generating efficient tiled code for distributed memory machines," *Parallel Computing*, 2000.

[59] M. M. Baskaran, U. Bondhugula, S. Krishnamoorthy, J. Ramanujam, A. Rountev, and P. Sadayappan, "Automatic data movement and computation mapping for multi-level parallel architectures with explicitly managed memories," in *Proceedings of the 13th ACM SIGPLAN Symposium on Principles and practice of parallel programming*, 2008.

[60] Y. N. Shen, R. C. Zhao, and L. Wang, "Eliminating redundant communication of code generation for distributed memory machines," in *Eighth ACIS International Conference on Software Engineering, Artificial Intelligence, Networking, and Parallel/Distributed Computing (SNPD 2007)*, 2007.

[61] U. Bondhugula, "Automatic distributed-memory parallelization and code generation using the polyhedral framework," IISc-CSA-TR-2011-3, Indian Institute of Science, Tech. Rep., 2011. [Online]. Available: https://www.csa.iisc.ac.in/~udayb/publications/uday11distmem-tr.pdf

[62] D. Narayanan, A. Harlap, A. Phanishayee, V. Seshadri, N. R. Devanur, G. R. Ganger, P. B. Gibbons, and M. Zaharia, "Pipedream: generalized pipeline parallelism for dnn training," in *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, 2019.

[63] M. Kruse, "Introducing molly: Distributed memory parallelization with llvm," arXiv preprint, 2014. [Online]. Available: https://arxiv.org/pdf/1409.2088

[64] R. Dathathri, "Compiling for a dataflow runtime on distributed-memory parallel architectures," Master's thesis, Indian Institute of Science, 2014, m.Sc. (Engineering) thesis.

[65] E. Slaughter, W. Lee, S. Treichler, M. Bauer, and A. Aiken, "Regent: a high-productivity programming language for hpc with logical regions," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2015.

[66] Z. Jia, M. Zaharia, and A. Aiken, "Beyond data and model parallelism for deep neural networks," Proceedings of Machine Learning and Systems (MLSys) 1, 2019. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2019/hash/b422680f3db0986ddd7f8f126baaf0fa-Abstract.html

[67] L. Zheng, Z. Li, H. Zhang, Y. Zhuang, Z. Chen, Y. Huang, Y. Wang, Y. Xu, D. Zhuo, E. P. Xing, J. E. Gonzalez, and I. Stoica, "Alpa: Automating inter- and intra-operator parallelism for distributed deep learning," 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22), 2022. [Online]. Available: https://www.usenix.org/conference/osdi22/presentation/zheng-lianmin

[68] M. Vaidya, S. Singh, D. Mantri, M. S. Eydenberg, B. M. Kelley, S. Rajamanickam, A. Rountev, and P. Sadayappan, "Distributed sparse tensor computations in mlir," in *Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2025.

[69] R. Yadav, A. Aiken, and F. Kjolstad, "Spdistal: Compiling distributed sparse tensor computations," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022.

[70] G. Bisbas, A. Lydike, E. Bauer, N. Brown, M. Fehr, L. Mitchell, G. Rodriguez-Canal, M. Jamieson, P. H. J. Kelly, M. Steuwer, and T. Grosser, "A shared compilation stack for distributed-memory parallelism in stencil dsls," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024.

[71] N. Stawinoga, D. Katz, A. Lydike, J. Zarins, N. Brown, G. Bisbas, and T. Grosser, "An mlir lowering pipeline for stencils at wafer-scale," in *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '26)*, 2026.

[72] Y. Huang, Y. Cheng, A. Bapna, O. Firat, M. X. Chen, D. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, and Z. Chen, "Gpipe: Efficient training of giant neural networks using pipeline parallelism," arXiv preprint (also NeurIPS 2019), 2018. [Online]. Available: https://arxiv.org/pdf/1811.06965

[73] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," arXiv preprint, 2019. [Online]. Available: https://arxiv.org/pdf/1909.08053

[74] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "Zero: Memory optimizations toward training trillion parameter models," in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2020.

[75] M. Jacquelin, M. Araya–Polo, and J. Meng, "Scalable distributed high-order stencil computations," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022.

[76] R. Baghdadi, J. Ray, M. B. Romdhane, E. D. Sozzo, A. Akkas, Y. Zhang, P. Suriana, S. Kamil, and S. Amarasinghe, "Tiramisu: A polyhedral compiler for expressing fast and portable code," in *2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2019.

[77] O. Project, "Shardy: an mlir-based tensor partitioning system (official documentation)," OpenXLA project documentation, 2024. [Online]. Available: https://openxla.org/shardy